Diffusion Cartograms in the Wolfram Language

A self-contained pure-Wolfram implementation of the Gastner–Newman density-equalising map algorithm.

Marco Thiel, April 2026 — github.com/mthiel74/Contiguous-Cartograms

Abstract

Cartograms are maps in which the area of each region is deformed to be proportional to a statistical quantity — population, GDP, number of COVID cases, votes cast, and so on — while the topology (who is next to whom) is preserved. Gastner and Newman (2004) showed how to produce such maps by interpreting the quantity as a density and letting it diffuse to uniformity; every point of the map is advected by the resulting velocity field $\boldsymbol{v} = -\frac{\nabla\rho}{\rho}$. This notebook presents a compact, pure-Wolfram-Language port of that algorithm. It ships as a paclet-style package **`CartogramWL`** exposing both low-level primitives (**`DiffusionSolver`**, **`Cartogram`**, **`CartogramTransform`**) and a one-liner **`WorldCartogram[metric]`** that pulls country polygons and statistical values from the built-in **`CountryData`** and returns a ready-to-publish figure. It supports any **`ColorData`** gradient, a **`GeoBackground -> "Satellite"`** mode that textures each country with deformed satellite imagery, user-supplied **`Association`** input, visualisation of the underlying space deformation via a distortion grid, and a **`PerformanceGoal -> "Speed"`** default that runs the full world pipeline in under a minute.

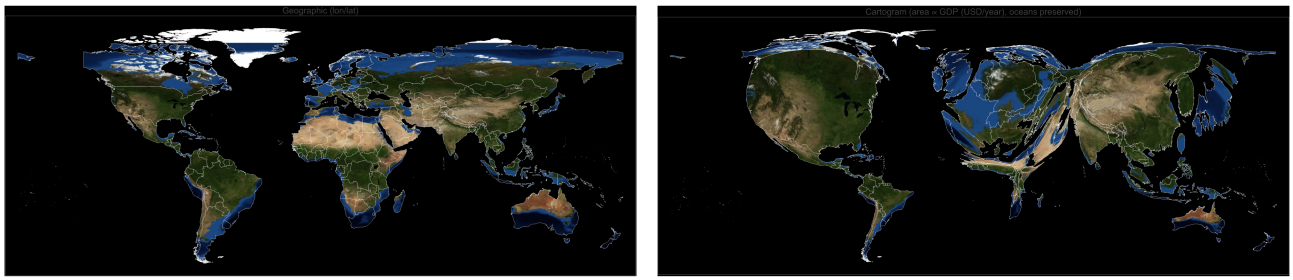


Figure 1. Left: countries on the geographic lon/lat grid, each textured with satellite imagery. Right: the same countries with area proportional to GDP, the satellite imagery deformed along with the geometry. Ocean area is preserved.

1. What is a cartogram?

A geographic map of the world devotes enormous screen area to Russia, Canada, and the Sahara, and a few pixels each to Bangladesh, South Korea, or the Netherlands. If the question being asked is "where do people live?" or "where is the economy?", this is the opposite of what we want. Den-

sity-equalising cartograms solve this by reshaping each region so that its area tells the story of a variable of interest.

Gastner & Newman's elegant insight (PNAS 2004) is that a map in which density $\rho(\boldsymbol{r})$ is uniform is exactly what we want, and diffusion is the natural process that takes any non-uniform density to a uniform one. We therefore simulate the heat equation

$$\partial_t\, \rho(\boldsymbol{r},\ t) = \nabla^2\, \rho(\boldsymbol{r},\ t), \quad \boldsymbol{n}\cdot\nabla\rho = 0, \quad \text{on } \partial\Omega$$

on a rectangular domain with Neumann boundary conditions (no flux through the box edge, so mass is conserved), and flow each geographic point through the instantaneous velocity

$$\frac{d\,\boldsymbol{r}}{d\,t} = \boldsymbol{v}(\boldsymbol{r},\ t) = -\frac{\nabla\ \rho(\boldsymbol{r},\ t)}{\rho(\boldsymbol{r},\ t)}$$

As $t \to \infty$ the density relaxes to its spatial mean and the advected points settle into their cartogram positions. Regions of high initial density draw space inwards and therefore expand; regions of low density are pushed outwards.

2. The algorithm, step by step

2.1 Rasterise polygons to a density grid

Given a collection of country polygons and a per-country metric M_c (population, GDP, …), we set the density inside polygon c to

$$\rho_0(\boldsymbol{r}) = \frac{M_c}{A_c}, \quad \text{for } \boldsymbol{r} \in c$$

where A_c is the geographic area of c. Integrated over c we recover the metric M_c. Ocean cells are filled with the mean land density so the spatial mean is uniform across the domain and oceans retain their size; only the relative sizes of countries change.

The rasterisation uses a `RegionMember`-based subpixel test for anti-aliasing on thin/small countries.

2.2 Solve the diffusion equation

The Laplacian on a rectangle with Neumann boundary conditions is diagonalised by the cosine basis. Wolfram's built-in `FourierDCT[…, 2]` and `FourierDCT[…, 3]` implement the DCT-II and its inverse in any number of dimensions with an orthonormal convention, so the full 2D spectrum is one call each. Expanding

$$\rho(\boldsymbol{r},\ t) = \sum_{m,n} \hat{\rho}_{m\,n}\, e^{-\lambda_{m\,n}\, t} \cos\left(\frac{\pi\, m\, x}{L_x}\right) \cos\left(\frac{\pi\, n\, y}{L_y}\right)$$

with eigenvalues $\lambda_{m\,n} = \pi^2 \left(\frac{m^2}{L_x^2} + \frac{n^2}{L_y^2}\right)$, the density at any time t is evaluated in $O(N\ \mathrm{Log}\ N)$ from the cached cosine coefficients $\hat{\rho}_{m\,n}$.

2.3 Advect points through the velocity field

The velocity on the grid is computed by central differences on the current diffused grid and interpolated bilinearly at any query point. Each geographic point is integrated from $t = 0$ to t^* using an

explicit Runge–Kutta scheme (`NDSolveValue[…, Method -> "ExplicitRungeKutta"]`) with a 5th-order stepper, where t^* is chosen from the decay constant of the slowest non-constant Fourier mode.

2.4 Pre-smoothing

Because polygon-rasterisation produces step functions, the initial velocity field is very stiff. We pre-smooth the density with a small Gaussian filter ($\sigma \approx 1$ cell), which is mathematically equivalent to starting the diffusion at $t_0 = \sigma^2 \, \Delta x \, \Delta y \,/\, 2$. A small background floor (`MeanFloor`) is added to avoid division by zero inside zero-density cells.

2.5 Snapshot-cached advection (the default Speed mode)

A naive implementation recomputes the full inverse DCT of $\rho(\boldsymbol{r},\ t)$ at every RK sub-step of the advection ODE. With ~100+ sub-steps on a 192×384 grid this dominates wall-clock at ~320 s for a single world map. The package avoids it: at construction time we pre-compute a small (default 60) set of geometrically-spaced snapshots of $(\rho,\ \partial_x \rho,\ \partial_y \rho)$, spanning $t = 0$ through t^* with more samples packed near $t = 0$ where the fast Fourier modes collapse. During integration the RHS linearly interpolates between the two bracketing snapshots instead of doing a fresh inverse DCT. Median per-point agreement with the Quality solver is sub-pixel at rendering resolution and the full world pipeline drops from ~5 min to ~45 s (~7× faster). Pass `PerformanceGoal -> "Quality"` to any function that integrates points (`AdvectPoints`, `CartogramRun`, `WorldCartogram`) for the slower, bit-exact variant.

---

3. The CartogramWL package

3.1 Loading

Clone the repository from GitHub and load the package directly from the `CartogramWL/` directory. The package has zero external dependencies.

```
Get["/path/to/wolfram/CartogramWL/CartogramWL.wl"]
```

All public symbols are then available in the `CartogramWL`` context:

```
Names["CartogramWL`*"]
```

3.2 Low-level primitives

The algorithm is exposed as a handful of composable functions that all operate on a density array `rho[[i, j]]` with row index $i \leftrightarrow y$, column index $j \leftrightarrow x$ (NumPy convention) and a bounding box `{xmin, ymin, xmax, ymax}`:

```
solver = DiffusionSolver[rho, {xmin, ymin, xmax, ymax}];
rhoAtT = DensityAt[solver, t];
newPts = AdvectPoints[solver, {{x1, y1}, {x2, y2}, ...}];
```

One level up, **Cartogram** bundles the pre-smoothing, mean floor, and ocean-preservation settings; **CartogramRun** precomputes the deformed cell-centre grid (Speed mode by default, ~7× faster than Quality); **CartogramTransform** maps arbitrary points through the deformation; **CartogramTransformPolygon** additionally densifies long edges before the transform.

```
cart = Cartogram[rho, bbox,
  "MeanFloor" → 0.02, "BlurSigma" → 1.5,
  "SeaDensity" → "auto"];
cart = CartogramRun[cart, PerformanceGoal → "Speed"];
newPoly = CartogramTransformPolygon[cart, polyRing];
```

3.3 The one-liner: WorldCartogram

Most end users want "given a metric, draw the world". **WorldCartogram** wraps the whole pipeline, reads polygons and values from **CountryData**, and returns a two-panel **GraphicsRow** (geographic + cartogram).

```
WorldCartogram["Population"]     (* or "GDP", "GDPPerCapita", ... *)
WorldCartogram["GDP", "ColorFunction" → ColorData["TemperatureMap"]]
WorldCartogram["GDP", "Background" → "Satellite"]
WorldCartogram[{"my metric", <|"Germany" → 83., ...|>}]
```

A complete option list is available via **?WorldCartogram**. The most useful options are **"ColorFunction"** (any **ColorData[...]** gradient or a raw function on $[0, 1]$), **"Background"** (**"Flat"** or **"Satellite"**), **"MissingCountries"** (**"Hide"** or **"ShowGrey"**), **"BoundingBox"** (useful for continent-scale maps), and **PerformanceGoal** (**"Speed"** default, or **"Quality"** for bit-exact output).

---

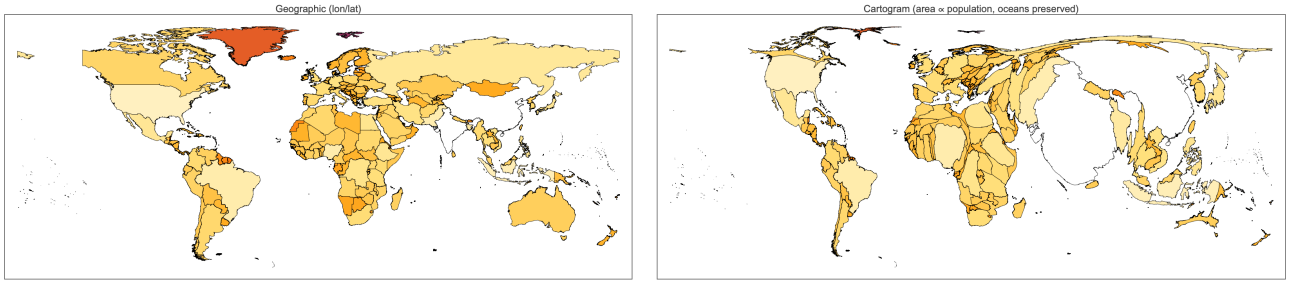
4. Built-in metrics

The four strings **"Population"**, **"GDP"**, **"GDPPerCapita"** and **"PopulationDensity"** are recognised directly and come with sensible defaults for the floor/ceiling that tame heavy-tailed outliers (Monaco on population density, Luxembourg on GDP per capita). The sections below show each of them in turn; every image on this page is produced by the literal one-line call shown next to it.

4.1 Population

The canonical cartogram: each country's area is made proportional to how many people live there. India and China together house ~36 % of humanity, so they swell enormously. Nigeria, Indonesia, Pakistan and Bangladesh together host nearly a billion more people across land areas that look small on a Mercator projection — all four bulge substantially. Russia (11 % of global land but 2 % of humanity), Canada, Australia, Mongolia and the Nordic countries collapse.

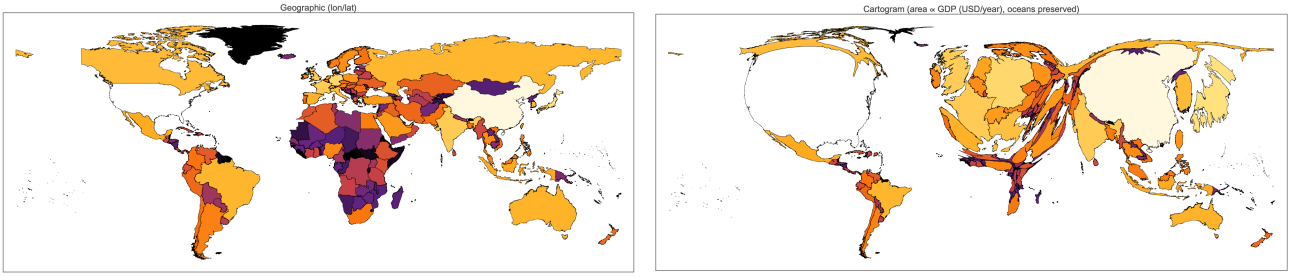
```
WorldCartogram["Population"]
```



4.2 GDP

Switching to gross domestic product (total value of goods and services produced in a year) rewards economic output rather than headcount. The United States, China, Japan, and the major western European economies dominate; sub-Saharan Africa and much of South Asia — densely populated but far poorer per head — shrink dramatically. This is the mirror image of the population cartogram wherever the two quantities diverge most: Japan (5 th largest economy, 11 th most populous) expands here but stays modestly sized in 4.1, while the Democratic Republic of the Congo does the opposite.

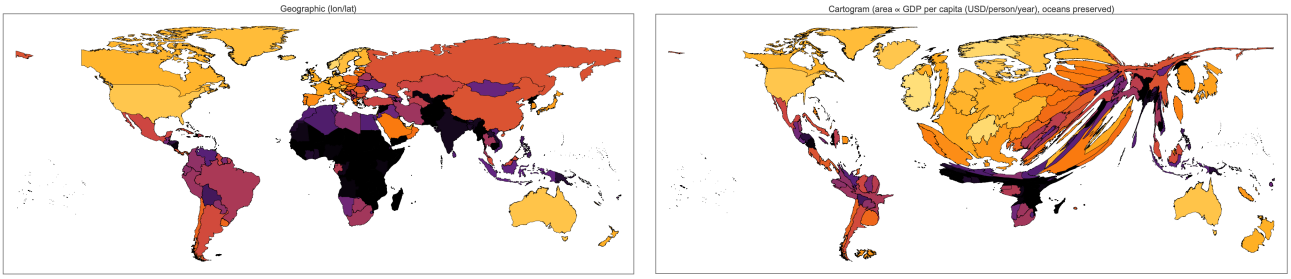
```
WorldCartogram["GDP"]
```



4.3 GDP per capita

GDP per capita (M_c = country GDP divided by country population) rewards wealth density rather than raw wealth. Luxembourg, Switzerland, Singapore, the Gulf petro-states, and the Scandinavian countries visibly bulge; sub-Saharan Africa, Central America, and parts of South Asia contract. Comparing this to the plain GDP cartogram in 4.2: China and India shrink here — they are large economies but not rich per head — while small high-income jurisdictions grow.

```
WorldCartogram["GDPPerCapita"]
```



4.4 Population density

Population density (people per km^2) asks a third question: not "where do people live?" but "where do people crowd?". Bangladesh, the Netherlands, Belgium, South Korea, and island states such as Singapore, Malta and Mauritius swell out of all proportion to their land area; the vast, sparsely populated interiors of Russia, Canada, Australia, Kazakhstan, and much of sub-Saharan Africa collapse. Unlike the absolute-population cartogram in 4.1, large countries like China and the United States end up modestly sized because their populations are diluted across a great deal of territory.

```
WorldCartogram["PopulationDensity"]
```

4.5 A detour: multi-part countries (Alaska, Hawaii)

Wolfram's `CountryData["UnitedStates", "Polygon"]` only returns the contiguous 48 states. Alaska and Hawaii live under `Entity["AdministrativeDivision", {"Alaska", "UnitedStates"}]` and its Hawaii analogue. If we rasterised the bare `CountryData` polygon the US would grow on the cartogram — but only the lower 48 would grow, because Alaska and Hawaii would contribute zero density to the diffusion field. `CartogramWL` therefore splices the administrative-division polygons back in via a helper `CountryPolygonRings[c]` before rasterising. Below is the full US ring set that actually feeds the algorithm — 28 mainland rings plus 77 Alaska rings (one ring per Aleutian island) plus 8 Hawaiian rings, 113 rings in total, all drawn in the same colour because they share a single country value. The same hook can be used to splice in France's overseas departments, Portugal's Azores/Madeira, Denmark's Greenland, and so on.

```
Length @ CountryPolygonRings["UnitedStates"]
(* 113  -- 28 mainland + 77 Alaska + 8 Hawaii *)
```

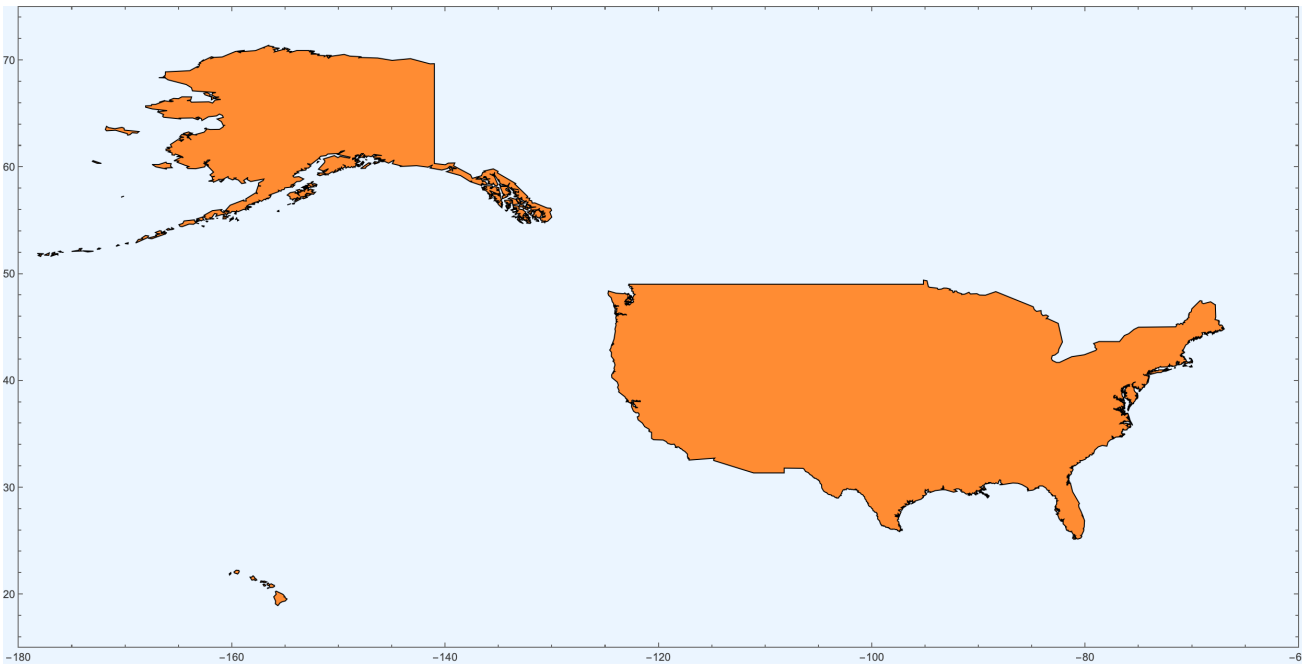
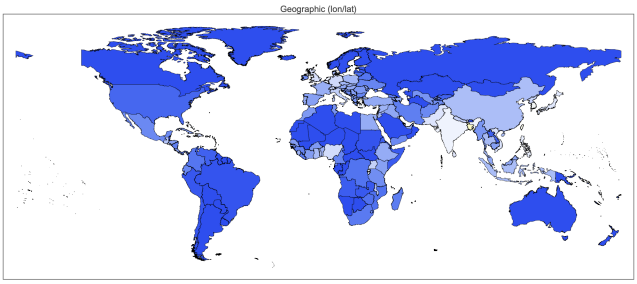


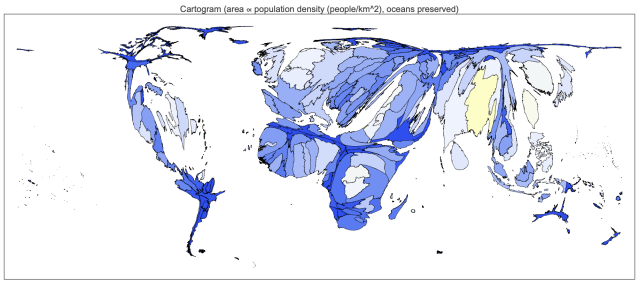
Figure 2. The polygon set that actually feeds the cartogram for the United States. Mainland (28 rings, right), Alaska mainland + Aleutians (77 rings, left), and Hawaii (8 islands, bottom-left). Every ring of the same country shares the country's per-area density, so Alaska and Hawaii grow alongside the mainland on the GDP or population cartogram.

5. Colour schemes & satellite background

Pass any `ColorData` gradient as `"ColorFunction"`:

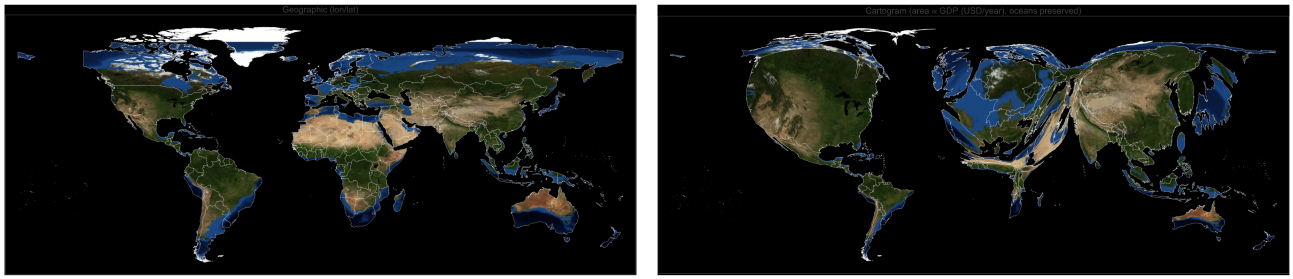
```
WorldCartogram["PopulationDensity",
 "ColorFunction" → ColorData["TemperatureMap"]]
```





Set `"Background" -> "Satellite"` and each country is textured with the corresponding patch of Wolfram's `GeoBackground -> "Satellite"` imagery. On the cartogram side the imagery is deformed along with the country geometry, giving an effect similar to a shaded-relief globe being inflated or deflated locally.

```
WorldCartogram["GDP", "Background" → "Satellite"]
```



6. Visualising the distortion itself

A clean way to see the geometry of the transformation is to overlay a regular 15° lat/lon grid on both panels. On the left the grid is undistorted; on the right the 325 grid intersections are pushed into the cartogram by the same velocity field that reshaped the countries. The grid therefore encodes the local stretching/compression factor at every point of the domain.

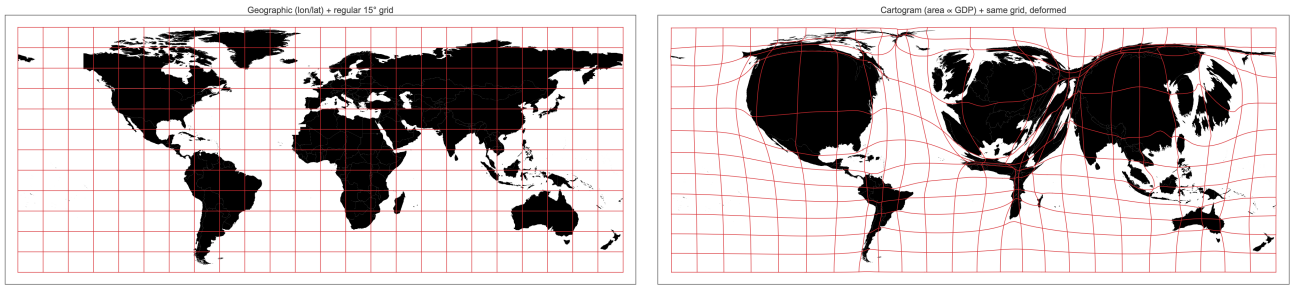


Figure 2. Left: geographic frame, regular 15° grid in red, countries grey. Right: the GDP cartogram, the same grid now warped by the same diffusion flow. Grid cells over high-wealth regions shrink (fewer dollars per square millimetre); cells over oceans and sparsely populated continents stretch.

`CartogramDistortionGrid[cart]` returns the `Line` primitives for the warped grid; wrap in `Graphics` to draw, overlay on a `WorldCartogram` result to annotate a cartogram with its own distortion field.

7. Bringing your own data

Any `Association` of country-identifier → value works in place of a metric string. Keys may be `CountryData` identifiers (e.g. `"UnitedStates"`) or display names (`"United States"`).

```
g20 = <|"UnitedStates" → 332., "China" → 1425.,
    "India" → 1428., "Japan" → 125., "Germany" → 83.,
    "UnitedKingdom" → 67., "France" → 67.5,  ...|>;

WorldCartogram[{"G20 population (millions)", g20},
 "MissingCountries" → "ShowGrey"]
```

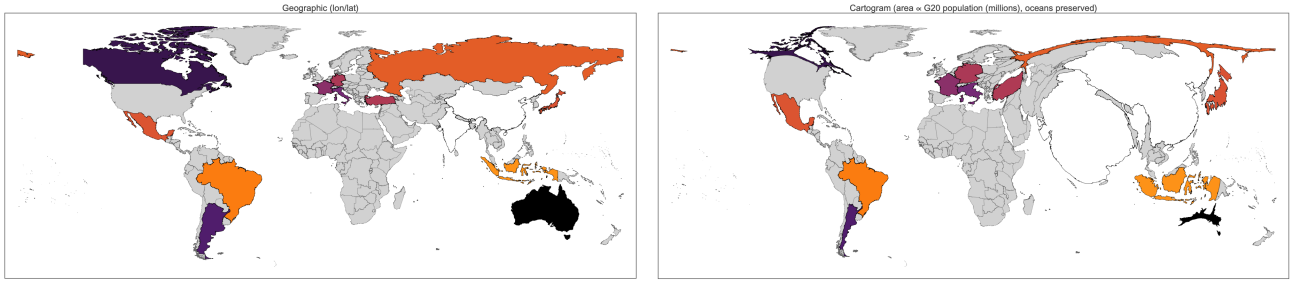


Figure 3. A cartogram driven by a hand-curated Association of 19 G20-ish economies; every other country appears in grey and does not contribute to the density field, but still visibly deforms because diffusion is global.

8. Implementation notes

• Only the diffusion is done in Fourier space; advection uses NDSolveValue's RK45 stepper over the reconstructed velocity. Modes below a relative tolerance (10^{-3} by default) are considered extinct, giving t^*.

• Speed mode replaces the per-sub-step inverse DCT with a linear interpolation between 60 log-spaced density/gradient snapshots, giving sub-pixel agreement with Quality at ~7× the throughput. Switch to `PerformanceGoal -> "Quality"` when you need bit-exact reproduction.

• Heavy-tailed metrics (Monaco on population density, Luxembourg on GDP per capita) produce enormous per-cell densities that can destabilise the advection. `WorldCartogram` clamps values at a configurable fraction of the mean and caps the rasterised grid at a multiple of the positive-cell mean.

• Long country edges are densified before being transformed, matching `shapely.segmentize` on the Python side of this project.

• The algorithm is grid-convention-compatible with the Python implementation so the two are drop-in interchangeable for debugging / cross-validation.

9. Related work in the Wolfram ecosystem

This is not the first Wolfram-Language treatment of cartograms. Two earlier resources are worth reading alongside this one:

• The 2019 Wolfram Community post Creating a contiguous cartogram for the US by George Varnavides presents a `Manipulate`-driven US-state cartogram with an interactive colour legend, originally written for the 2019 Wolfram Summer School. It focuses on a single country, uses an iterative scaling update rather than diffusion, and is a nice demonstration of how much can be done inside a notebook with no package at all.

• `ResourceFunction["Cartogram"]` — resources.wolframcloud.com/FunctionRepository/resources/Cartogram — is a function-repository entry that produces a contiguous cartogram directly

from an `EntityClass` plus a property. It is a single-call convenience similar in spirit to `WorldCartogram`, runs inside the Wolfram Cloud with no installation, and is the right tool when you need a quick cartogram without thinking about parameters.

Relative to those two, the contribution of the present package is (a) a pure-Wolfram port of the full Gastner–Newman diffusion algorithm exposed as a handful of composable primitives (`DiffusionSolver`, `AdvectPoints`, `RasterizePolygons`, …) that can be used on any rectangular domain, not just a country set; (b) built-in support for ocean-area preservation, user-supplied `Association` input, and satellite-textured rendering; and (c) a default `PerformanceGoal -> "Speed"` mode that cuts a full world cartogram from ~5 minutes to under a minute while staying sub-pixel-accurate.

10. References

- M. T. Gastner & M. E. J. Newman, "Diffusion-based method for producing density-equalizing maps", *PNAS* **101**(20), 7499–7504 (2004). doi:10.1073/pnas.0400280101.
- Source code and the Python twin:
  github.com/mthiel74/Contiguous-Cartograms.
- Relies on Wolfram's `CountryData`, `GeoGraphics`, `FourierDCT`, and `NDSolveValue`.